

# Campus Radar

## Higher-Education Performance Intelligence Platform

A portfolio case study

### William Elturk

Senior Full-Stack Developer / Technical Lead

Frontend Architecture · SQL & Analytics Engineering · Power BI

Next.js 15 / React / TypeScript · C# / .NET 8 · EF Core · SQL Server · Hangfire · Redis · amCharts 5 · Tailwind / MUI · Active Directory SSO · Azure DevOps · xUnit

The client is a national higher-education institution in the Gulf region. To protect confidentiality, the client name, environments and data have been anonymized. All screenshots in this case study were captured from a local development environment using synthetic data.

# 1 Context & Challenge

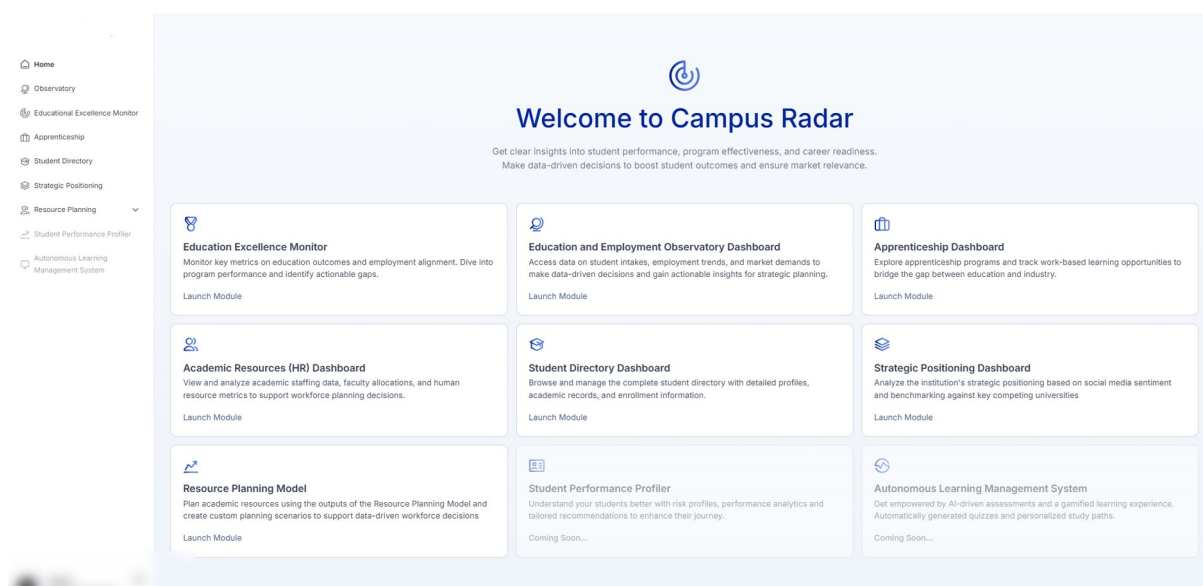
The client operates a multi-campus network of applied-technology colleges. Leadership needed to track both internal educational performance and a national ministry performance framework. This included measures such as curriculum alignment with labour-market needs, work-based learning, retention and graduation, graduate employment, starting salary, research performance and accreditation.

The information needed to calculate these indicators came from several sources, including student records, faculty and course data, apprenticeship and work-placement records, employer feedback and finance. Reporting was fragmented and often required manual spreadsheet work. Resource planning was also handled separately. Campus Radar was created to bring these areas into one platform, giving executives an institution-wide view, campus leaders a focused view of their own campus, and planners the ability to model future staffing and budget scenarios.

|                                                         |                               |                                             |                                   |                                              |
|---------------------------------------------------------|-------------------------------|---------------------------------------------|-----------------------------------|----------------------------------------------|
| 111<br>stored procedures<br>— KPI and<br>planning logic | 114<br>typed API<br>endpoints | 151<br>API actions across<br>15 controllers | 321<br>automated<br>backend tests | 3<br>environments<br>with branch-based<br>CI |
|---------------------------------------------------------|-------------------------------|---------------------------------------------|-----------------------------------|----------------------------------------------|

## What the platform does

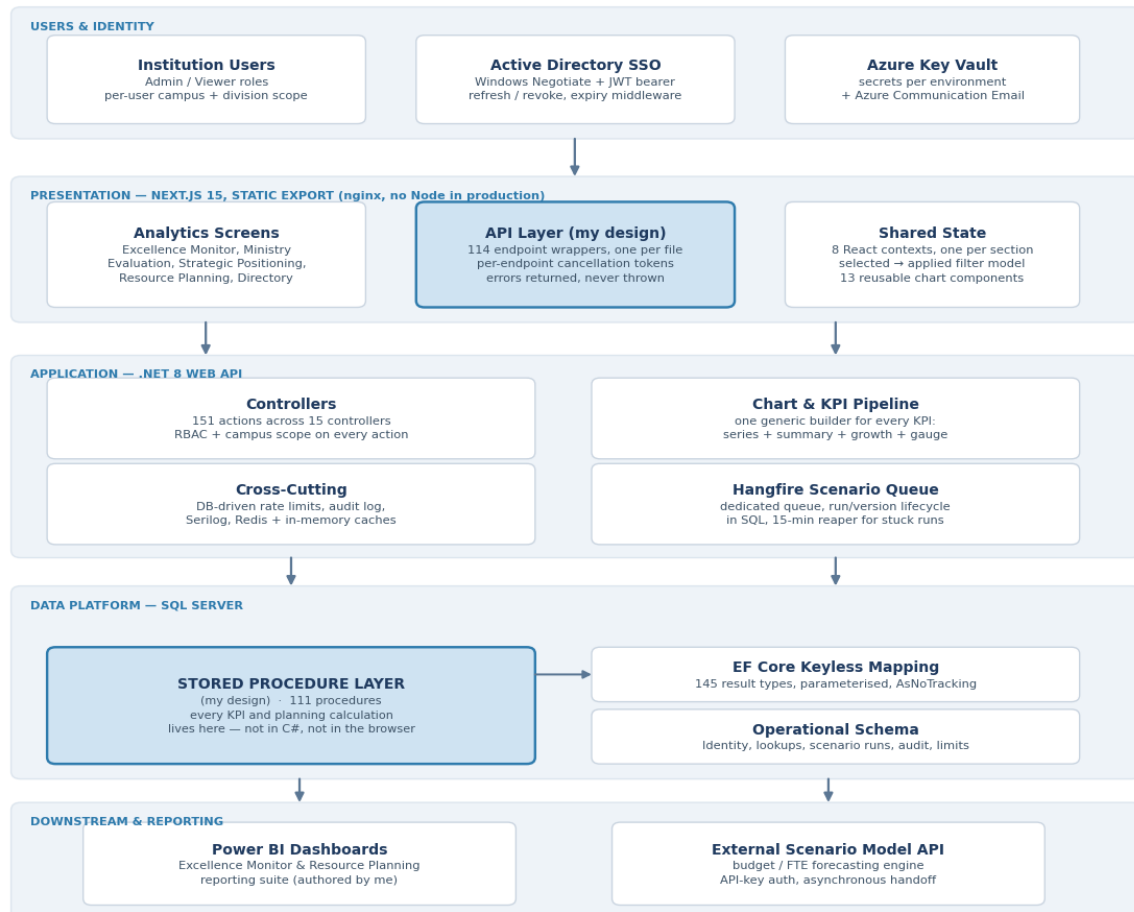
- Educational Excellence Monitor — Tracks the main academic performance indicators across inputs and outcomes, including programme alignment, competency-based teaching, practical learning, class size, student-faculty ratios, faculty experience, retention, graduation and graduate employment.
- Ministry evaluation — Tracks the national higher-education performance framework in the same platform, including research, citations, international collaboration, accreditation, rankings, and community and industry engagement.
- Strategic positioning & personas — Provides student-level analysis through cohort personas, performance tiers, at-risk identification and drill-down from summary charts to student lists and individual profiles.
- Resource planning — Models FTE demand and supply, staffing gaps and budget forecasts by campus, division, education level and position. Scenarios are versioned, processed through an external forecasting model and can be compared side by side.
- Apprenticeship & student directory — Covers placement effectiveness, assessor-to-student ratios, mentor nomination and employer feedback.
- Administration — Uses Active Directory-backed users and roles, with campus and division scope applied per user across the platform.



Home — Executive entry point and module launcher

## 2 Architecture

The solution uses three main layers. A key architectural decision was to keep business calculations in the database. The frontend is a Next.js static export served by nginx, so no Node.js runtime is required in production. The .NET 8 API handles authentication, authorization, user scope and response shaping, while KPI and planning calculations are produced by stored procedures. Procedure results are mapped into EF Core as keyless result types and exposed through the API. The frontend is responsible for presenting the results rather than reimplementing business logic.



Solution architecture. The highlighted areas are the stored-procedure layer and the frontend API integration layer, which I designed and led.

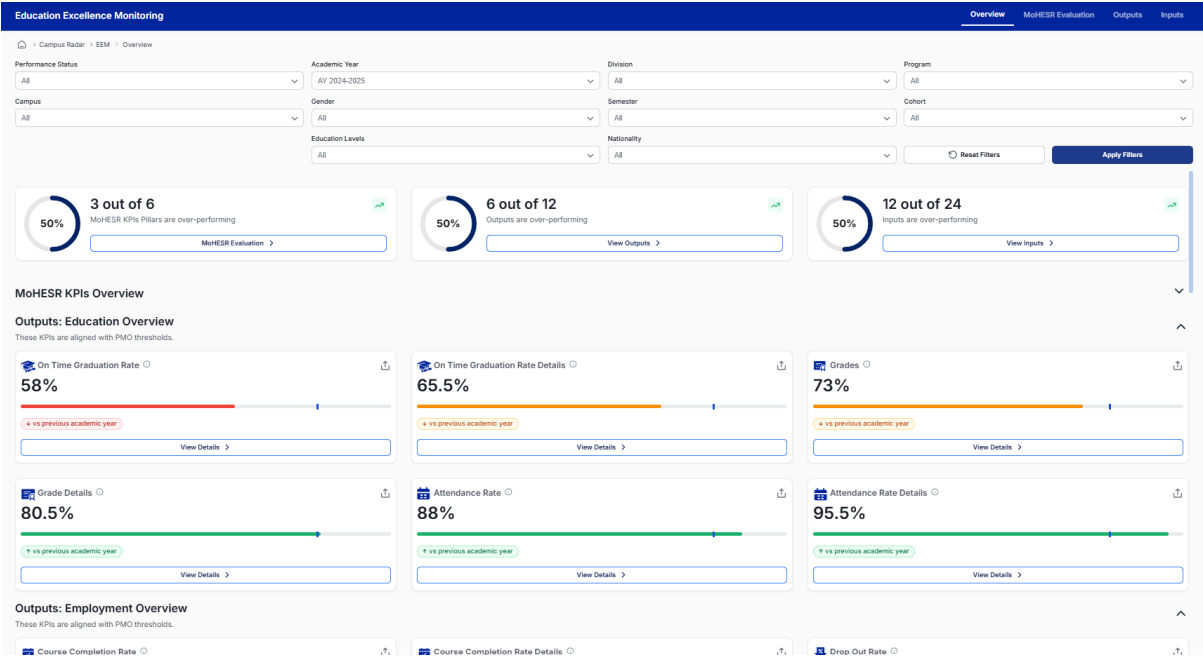
### Key architecture decisions

- A single source for calculations. Each KPI is defined once in a stored procedure and used by both the application and Power BI. This avoids duplicating business logic in C# or JavaScript and keeps figures consistent across reporting channels.
- Static frontend delivery. Using Next.js static export produces a deployable set of static files served by nginx. This keeps the production frontend simple while authentication and protected operations remain behind the API.
- Consistent implementation patterns. The API wrappers and KPI procedures follow repeatable conventions. This reduced variation across 114 endpoints and made it easier for developers to add new functionality without introducing a different pattern for every page.
- Configuration stored as data. Rate-limit policies and visual thresholds are stored in database tables and loaded by the application, allowing selected business and operational settings to be changed without a deployment.

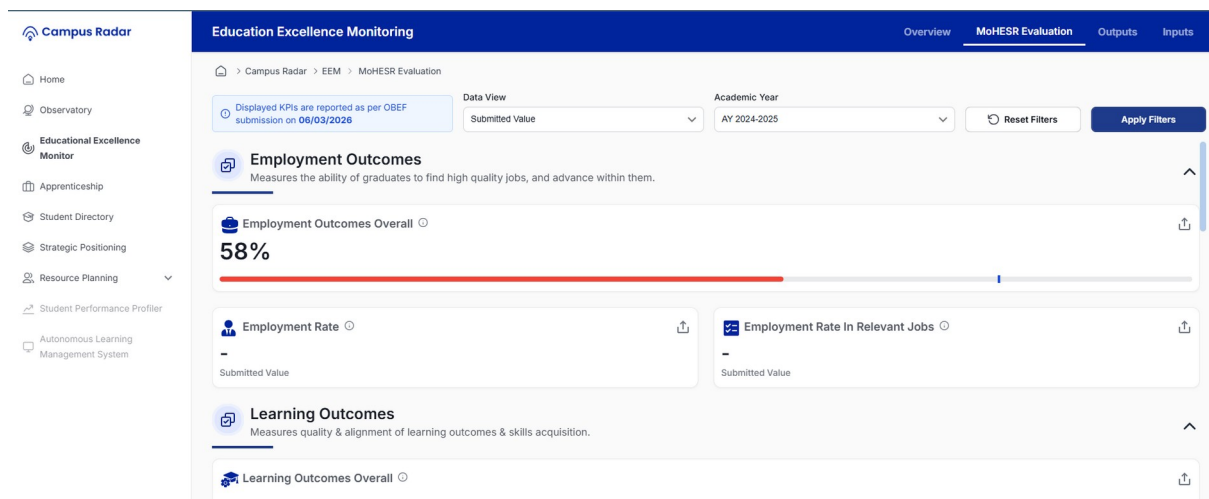
### 3 My Role | Scope of Contribution

The platform was delivered by an agile team across several workstreams. My contribution focused mainly on frontend architecture, the analytical SQL layer, selected .NET backend areas and Power BI. The breakdown below separates the areas I led from the areas where I contributed as part of the wider team:

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LED            | <p>Frontend architecture — Led the Next.js application architecture, including route and folder conventions, the 114-wrapper API integration layer, request cancellation, error handling, the two-stage filter model, reusable chart and drill-down components, and static-export delivery behind nginx. I also wrote the frontend engineering guide used for onboarding.</p> <p>Stored-procedure layer — Designed the 111-procedure analytical layer covering KPI definitions and resource-planning calculations, together with the shared parameter conventions used by the API.</p> |
| CONTRIBUTOR    | .NET backend for the Excellence Monitor and Resource Planning sections — Contributed to controllers, DTOs, the shared chart/KPI builder and EF Core mappings that expose stored-procedure results through the API.                                                                                                                                                                                                                                                                                                                                                                     |
| ALSO DELIVERED | Power BI dashboards for the Excellence Monitor and Resource Planning sections, built against the same stored-procedure layer used by the web application.                                                                                                                                                                                                                                                                                                                                                                                                                              |



Educational Excellence Monitor — KPI families with performance bands and a shared filter model



Ministry evaluation — National performance indicators integrated into the same portal

## 4 Deep Dive | The stored-procedure layer (data & calculation architecture)

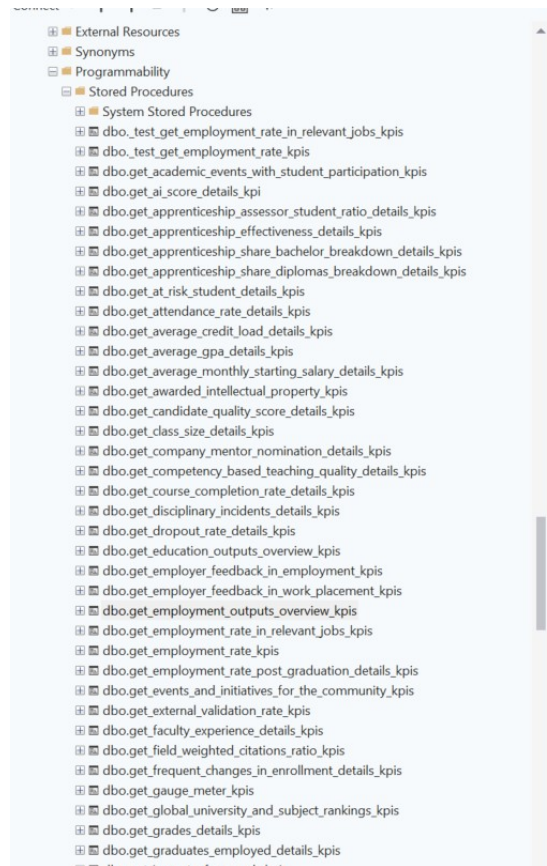
The stored-procedure layer is the analytical foundation of the platform. It contains 111 procedures: 77 for KPI calculations and 34 for resource-planning calculations. My focus was not only on writing the SQL, but on defining a consistent contract that could support 114 endpoints, 145 result types and the related automated tests without duplicating calculation logic across the application.

### The contract

- One procedure per indicator, with a shared signature. KPI procedures use the same ten-filter parameter set: programme, gender, academic year, campus, division, cohort, semester, performance status, nationality and education level. Filters are passed as comma-separated ID lists, with an empty value meaning no filter. This consistency allowed the backend and frontend to reuse the same request patterns across the KPI set.
- Scenario-aware planning procedures. Resource-planning procedures accept scenario IDs together with semester, campus, division, faculty type, course type, education level and job title. Baseline and forecast scenarios therefore use the same calculation path and differ mainly through their parameters.
- EF Core result mapping. Procedure result sets are mapped to keyless EF Core types and exposed as `IQueryable` results where additional projection or aggregation is needed. Parameters are passed using parameterized SQL and read-only queries are untracked.
- Easy to validate independently. Procedures use default parameters and suppress row-count messages, so they can be executed directly in SQL Server when a KPI needs to be checked against an application or Power BI result.
- Thresholds stored in the database. Gauge ranges and progress-bar thresholds are maintained as data rather than hard-coded values, making them easier to adjust as business definitions change.

### What the procedures actually compute

The procedures cover the main academic and planning domains of the platform. These include curriculum alignment with industry needs, teaching quality, practical and work-based learning, student attendance and outcomes, retention and graduation, at-risk identification, graduate employment, role relevance, time to first job and starting salary. The ministry indicators add research, accreditation, rankings and engagement measures. Resource-planning procedures calculate FTE demand and supply, staffing gaps and budget forecasts across campus, division, education level, position, faculty type and course type.



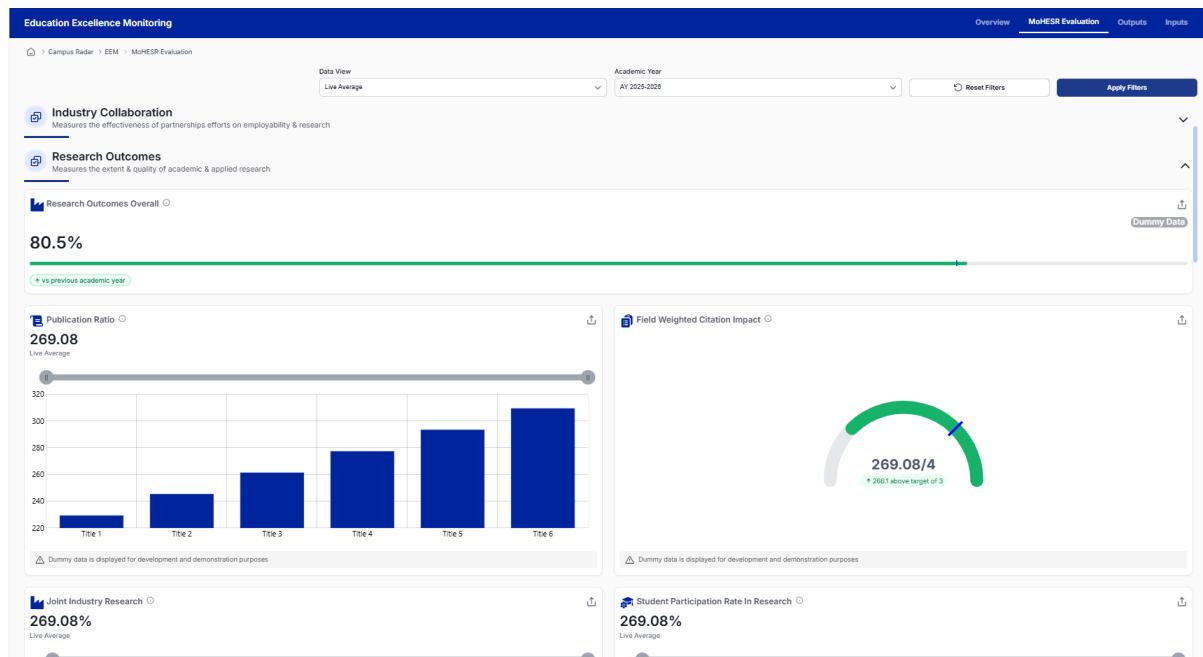
Part of the stored-procedure layer — individual procedures are organized around the KPI or planning calculation they provide

## 5 Deep Dive | A front end that computes nothing

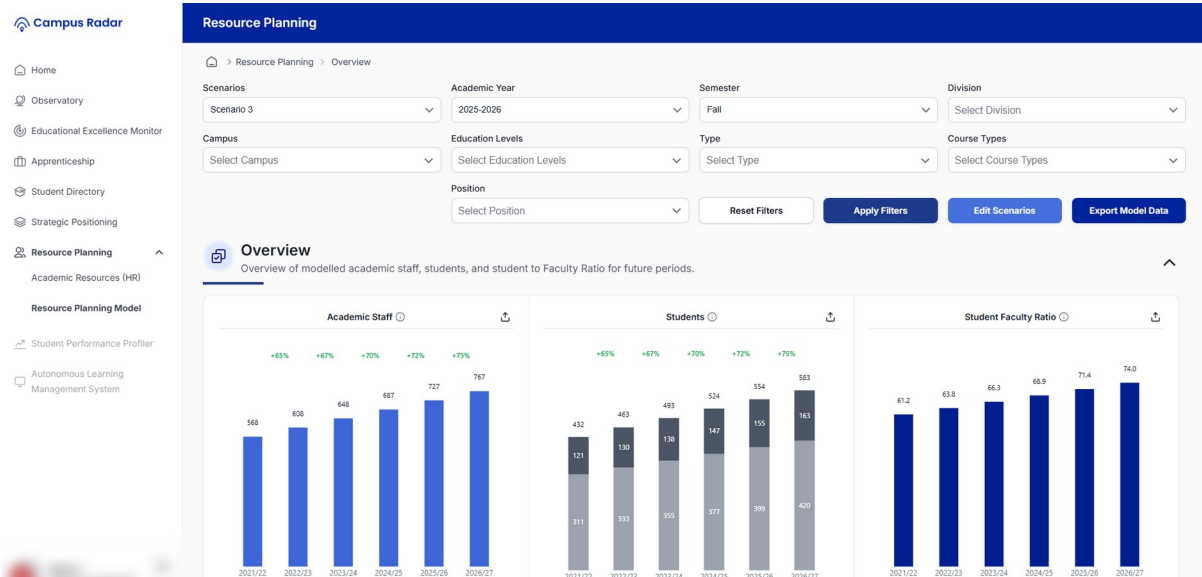
A core frontend principle was to keep business calculations out of the browser. With 114 endpoints, consistency mattered more than creating page-specific patterns. I defined and documented the conventions used for API access, filter state, request cancellation, error handling and reusable visual components.

- One wrapper per endpoint. Each endpoint has a dedicated wrapper that reads its URL from a central endpoints map and calls a shared Axios instance responsible for authentication and token refresh. Application components do not call Axios directly.
- Request cancellation. Each wrapper can cancel its previous request, which is useful on highly interactive pages where filters can change quickly. This prevents outdated responses from replacing newer results.
- Predictable error handling. API wrappers return a consistent error/result structure, and cancelled requests are handled as an expected state rather than being shown to the user as application errors.
- Two-stage filter state. Each section separates selected filter values from applied values. Pages only reload when the user applies the filters, avoiding unnecessary requests while the user is still making selections.
- Reusable response and chart patterns. KPI endpoints return a small number of response shapes, allowing a shared set of chart components and drill-down tables to be reused across the platform.

There are also trade-offs in this design. Because cancellation is managed at endpoint-wrapper level, two components calling the same endpoint at the same time could cancel one another; in the current page structure, each page owns its requests, so this is acceptable. The wrapper barrel export is also a manual step and can be missed when adding a new endpoint. Both cases are documented in the frontend engineering guide so developers understand the convention and its limitations.



Reusable visual components — progress bands, distribution charts and threshold gauges built on shared response shapes



Resource Planning — scenario-based staffing, student and ratio projections using the shared filter model

## 6 Deep Dive | Resource-planning scenarios as a queued pipeline

Resource planning is not handled as a long-running HTTP request because the forecasting model runs as a separate service and can take several minutes. Instead, scenarios and their versions are stored as persistent records in SQL. When a user submits a run, the application queues a background job, which authenticates to the external forecasting service, passes the run identifier and returns control to the application. The model then writes results back against that run. The interface reads the persisted status, so it can show progress without keeping a request open.

Failure handling was built into the same flow. A recurring job checks for runs that remain in a running state beyond a defined timeout and marks them as failed in a set-based update. This prevents stalled scenarios from remaining active indefinitely. Because versions are stored separately, planners can compare previous and current forecasts without overwriting earlier results.

## 7 Deep Dive | Access control and abuse protection

Because the platform contains student-level information, access control is applied at several layers. Internal users can authenticate through Windows/Active Directory single sign-on, while the static frontend uses JWT bearer tokens with refresh and revocation support. Middleware also ensures that requests have an identifiable user or IP address so that auditing and rate limiting can be applied consistently.

- Role and data scope. Controller actions enforce roles, while campus and division assignments are resolved server-side for each user and cached. The same scope is then applied to filters, charts and drill-down results.
- Database-driven rate limiting. Global and hourly policies are loaded from configuration tables, and rejected requests include a `Retry-After` header.
- Request auditing. Controller requests are logged and written to an API audit trail, with structured application logs providing additional operational context.

## 8 Engineering practice

- 321 automated backend tests cover the areas most likely to fail as the application changes, including EF Core entity configuration, stored-procedure mapping and parameter coverage, controller behavior, shared KPI/chart services and user filter caching.
- Azure DevOps pipelines are configured for both repositories across development, staging and production. The frontend selects its environment configuration by branch and publishes the static bundle, while the .NET API is built and packaged for IIS deployment.
- Frontend engineering guide. I documented the folder structure, page anatomy and endpoint-creation workflow so new developers could follow the same conventions without relying on informal knowledge from the existing team.

|                                              |     |
|----------------------------------------------|-----|
| EemOverview                                  | ▼   |
| Filter                                       | ▼   |
| Inputs                                       | ▼   |
| Mohestr                                      | ▼   |
| Outputs                                      | ▼   |
| RateLimit                                    | ▼   |
| RPAAnalysis                                  | ▼   |
| RPFfilter                                    | ▼   |
| SppAnalysis                                  | ▼   |
| SppPersona                                   | ▼   |
| SppStudentsList                              | ▼   |
| Users                                        | ^   |
| GET /api/users/getCurrentUser                | 🔒 ▼ |
| POST /api/users/refreshToken                 | ▼   |
| POST /api/users/revokedToken                 | ▼   |
| GET /api/users/validateToken                 | 🔒 ▼ |
| POST /api/users/uploadProfileImage/{id}      | 🔒 ▼ |
| GET /api/users/getProfilePhoto/{id}          | 🔒 ▼ |
| POST /api/users/register                     | ▼   |
| POST /api/users/updateUserStatus             | 🔒 ▼ |
| POST /api/users/deleteUsers                  | 🔒 ▼ |
| POST /api/users/updateUsersInfo              | 🔒 ▼ |
| GET /api/users/getUsers                      | 🔒 ▼ |
| GET /api/users/getActiveCampusesAndDivisions | 🔒 ▼ |
| UsersAD                                      | ^   |

API surface — controllers organized by functional area, with authorization applied to protected endpoints

## 9 Outcomes

### Platform Outcomes

- Combined academic performance, graduate outcomes, student risk, apprenticeship and resource planning in one governed portal.
- Used the same SQL calculation layer for both in-app analytics and Power BI, keeping KPI definitions consistent across reporting channels.
- Supported both institution-wide and campus-level analysis, with each user's campus and division scope applied consistently across filters, charts and drill-downs.
- Integrated the national ministry performance framework with the institution's internal performance measures in the same platform.
- Introduced versioned resource-planning scenarios so planners could compare forecasts without overwriting previous assumptions and results.
- Improved operational governance through role-based access, data scoping, request auditing, configurable rate limiting and handling for stalled forecasting runs.

### My Technical Contribution

- Led the frontend architecture, including route and folder conventions, the 114-endpoint API integration layer, shared filter state, reusable chart and drill-down components, and static-export deployment.
- Designed the stored-procedure layer containing 111 KPI and resource-planning procedures, together with the parameter conventions used by the API.
- Standardized KPI filter parameters so the same request and integration patterns could be reused across the backend and frontend.
- Translated academic performance and resource-planning rules into SQL calculations used by both the application and Power BI dashboards.
- Contributed to the .NET backend for the Excellence Monitor and Resource Planning areas, including controllers, DTOs, shared KPI/chart services and EF Core mappings for procedure results.
- Implemented the frontend request-cancellation, error-handling and two-stage filter patterns used across the analytics sections.
- Built Power BI dashboards for the Excellence Monitor and Resource Planning sections using the same SQL calculation layer as the web application.
- Created a frontend engineering guide to document the architecture and help onboard developers to the codebase.

### What this project says about me

Campus Radar reflects the type of full-stack work I enjoy most: understanding the business rules, deciding where those rules should live, and then building a clear contract between the data, API and frontend layers. My main ownership was across the frontend architecture and SQL analytics layer, while I also contributed to the .NET backend and Power BI delivery. The project shows how I approach larger applications: keep business logic consistent, use repeatable engineering patterns, document the important decisions, and make the system easier for the next developer to extend.

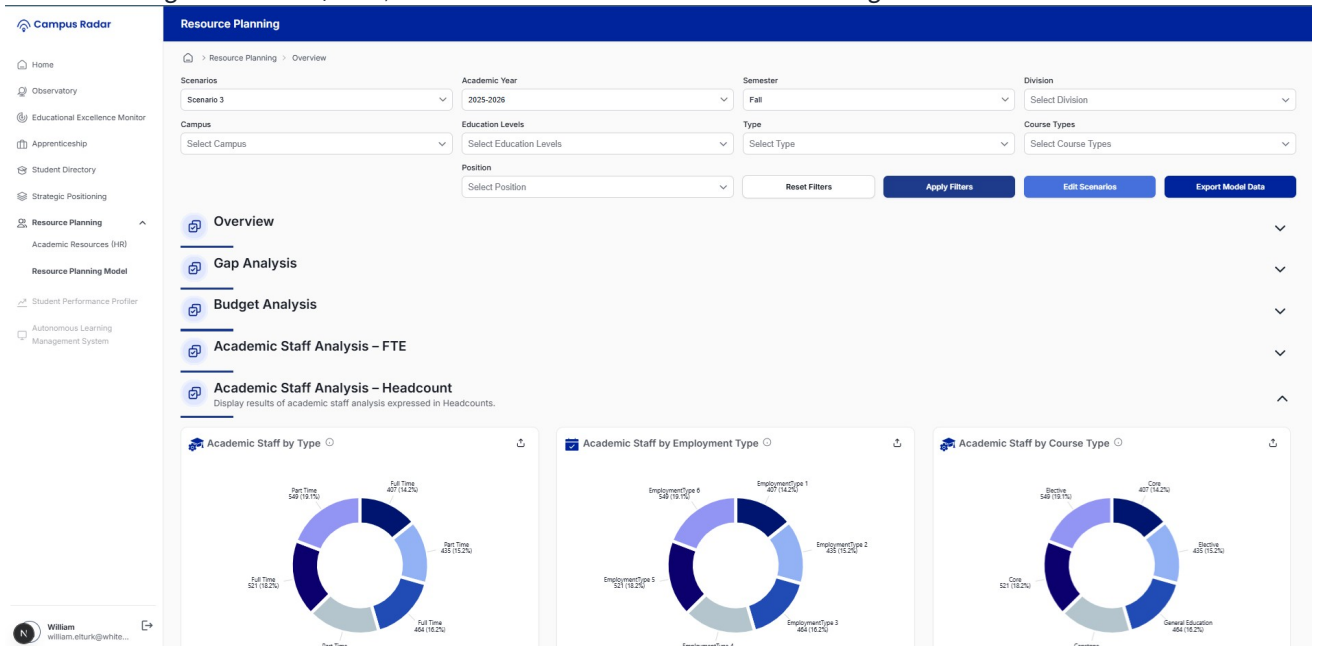
---

William Elturk — Senior Full-Stack Developer / Technical Lead · Frontend Architecture · SQL & Analytics Engineering · Power BI  
All names, data and environments in this document are anonymized. Screenshots show a local development environment using synthetic data.

# 10 Gallery

## SHOT-08

Resource Planning Overview — filters, sections and academic staff distribution widgets



SHOT-09

Gap Analysis — scenario comparison across staffing, students and student-faculty ratio



## SHOT-10

Export Data — scenario-level model output prepared for export and downstream analysis

Scenario: Scenario 3 Academic Year: 2025-2026 Semester: Fall

| Scenario   | Academic Year | Semester | Education Level | Campus      | Division                     | Faculty Type | Employment Type  | Course Type       | Grade   | Job Title           | Headcount | FTE Staff | Budget     | Student Enrolment | Enrolment Gov. Funded | Enrolment Non-Gov. Funded |
|------------|---------------|----------|-----------------|-------------|------------------------------|--------------|------------------|-------------------|---------|---------------------|-----------|-----------|------------|-------------------|-----------------------|---------------------------|
| Scenario 3 | 2025-2026     | Fall     | Diploma         | Riverside   | Engineering Technology       | Full Time    | EmploymentType 1 | Core              | Grade 1 | Professor           | 553.88    | 527.34    | 10,531,500 | 249.9             | 314.32                | 122.24                    |
| Scenario 3 | 2025-2026     | Fall     | Higher Diploma  | Harbourview | Business                     | Part Time    | EmploymentType 2 | Elective          | Grade 2 | Associate Professor | 592.63    | 564.25    | 11,266,705 | 267.39            | 336.32                | 130.79                    |
| Scenario 3 | 2025-2026     | Fall     | Bachelor        | Westbridge  | Health Sciences              | Full Time    | EmploymentType 3 | General Education | Grade 3 | Assistant Professor | 631.4     | 601.17    | 12,005,910 | 284.89            | 358.33                | 139.35                    |
| Scenario 3 | 2025-2026     | Fall     | Masters         | Northgate   | Computer Information Science | Part Time    | EmploymentType 4 | Capstone          | Grade 4 | Lecturer            | 670.17    | 638.08    | 12,743,115 | 302.38            | 380.33                | 147.91                    |
| Scenario 3 | 2025-2026     | Fall     | Doctorate       | Lakeside    | Applied Media                | Full Time    | EmploymentType 5 | Core              | Grade 5 | Instructor          | 708.94    | 675       | 13,480,320 | 319.87            | 402.33                | 156.47                    |
| Scenario 3 | 2025-2026     | Fall     | Bachelor        | Eastfield   | Education                    | Part Time    | EmploymentType 6 | Elective          | Grade 6 | Teaching Assistant  | 747.71    | 711.91    | 14,217,525 | 337.37            | 424.34                | 165.02                    |